UniKey: Keypoints as a Unified Interface for Scalable Data Generation and Policy Learning

Yuan Fang[1], Yixuan Wang[1], Peng-Hao Hsu[1], Richard Cheng[2], Changxi Zheng[1], Katherine Liu[2], and Yunzhu Li[1]

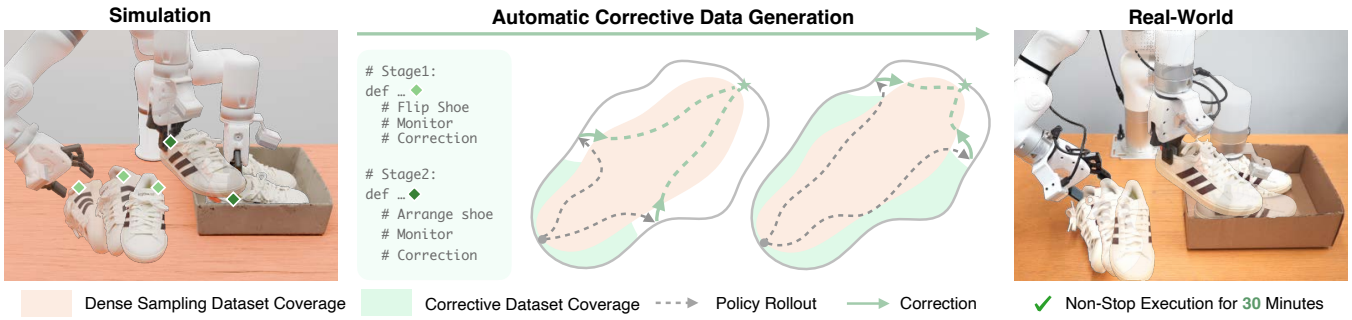


Fig. 1: **UniKey: Automatic Corrective Data Collection for Robust Imitation Learning** Achieving robust imitation learning policy in real world typically requires humans to repeatedly evaluate policy and collect corrective data. Our framework automates this corrective data generation process by using keypoints as a unified interface for policy monitoring and motion planning. During simulated rollouts, keypoints are used to detect policy deviations and generate corrective data automatically. Policies are iteratively evaluated and fine-tuned using the collected corrective data. Trained entirely in simulation, policies achieve near-perfect success rates in real world.

***Abstract*— Imitation learning policies are prone to compounding errors during rollout. Improving robustness typically requires multiple iterations of policy evaluation and human corrective demonstrations around failure regions, which is costly and difficult to scale. In this work, we introduce UniKey that uses keypoints as a unified interface for policy monitoring and motion planning, which automates corrective data collection, enabling robust policy performance. We formulate manipulation tasks as an optimization problem over relational keypoints. Additionally, we reuse the same set of keypoints to monitor task progress and detect policy deviations during rollout. Once a failure is detected, the motion planner takes over from the deviated state and generates corrective demonstrations, which are aggregated into the training dataset for policy finetuning. We evaluate our framework on three sim-to-real tasks and ten tasks from the RoboTwin benchmark. Compared with dense sampling, our approach achieves 2.65× higher data efficiency and near-perfect policy performance in both simulation and real world. Website: https://anonymous.4open.science/w/sdk-review-91E2/**

Keywords: Imitation learning, Failure Detection and Recovery, Manipulation Planning

I. Introduction

Imitation learning policies [1–3] are susceptible to compounding errors: small deviations from expert behavior can lead to states poorly covered by the training data, where further errors become more likely and recovery becomes difficult. Improving robustness often involves repeatedly evaluating the policy and augmenting the training data with human-provided corrections [4]. However, relying on humans to identify failures and demonstrate recovery behaviors makes this process costly and difficult to scale. Automating this corrective data collection loop is therefore an important step toward scalable robust manipulation policies.

We introduce UniKey, a framework that uses *keypoints as a unified interface* to automatically (1) plan motion to solve the task, (2) monitor policy progress, and (3) generate corrective data. Keypoints provide a unified geometric interface between low-level simulator states and high-level manipulation tasks, enabling both motion planning and policy monitoring. Our framework uses a vision-language-model agent to generate multi-stage keypoint programs. During policy execution, keypoint programs monitor policy task progress at the stage level and end-effector keypoint deviations at the step level. Once a failure is detected, the planner takes over from the failure state and automatically generates corrective motions. The resulting corrective data is aggregated into the training dataset, and the policy is finetuned on the dataset to improve robustness.

We identify *divergence detection* as the key for corrective data generation. Our framework monitors policy failures at both the stage and step levels. First, it uses the keypoint constraints to evaluate the policy trajectory and identify the failed stage. Then, it compares the end-effector keypoint trajectory against the planner trajectory to locate the earliest divergent step. The simulator is then reset to the state at this step, and the planner takes over control to generate corrective motions. By identifying both the timestep at which divergence begins and what stage the planner should

[1]Department of Computer Science, Columbia University, New York, NY, USA.
[2]Toyota Research Institute, USA.

accomplish next, our framework enables automatic corrective data generation.

We compare our framework against dense sampling on three sim-to-real manipulation tasks and ten tasks from the RoboTwin2.0 benchmark [5] on three mainstream imitation-learning policies: ACT [1], Diffusion Policy (DP) [2], and $\pi_{0.5}$ [3]. Both approaches start from the same initial dataset and add an equal amount of data at each iteration, followed by policy fine-tuning and rollout evaluation. Across these tasks, our method achieves $2.65\times$ the data efficiency of dense sampling, and nearly perfect execution performance in the real world. Our website lists uncut videos of the policy continuously doing tasks for 30 minutes.

In summary, our contributions are threefold:

- We introduce a keypoint-program framework for expert and corrective data generation. By formulating manipulation tasks as keypoint-constrained optimization problems, our framework supports motion planning from both initial task states and policy failure states.
- We develop an automatic divergence detection method that enables scalable and consistent evaluation of policy execution at both the stage and step levels, without relying on human judgment. It identifies the failed stage using keypoint constraints, then compares the end-effector keypoint trajectory with the planner reference to locate the earliest divergent step.
- We evaluate our framework against dense sampling on three sim-to-real manipulation tasks and ten simulation tasks with three imitation-learning policies. Our framework achieves $2.65\times$ the data efficiency of dense sampling and nearly perfect performance in real world.

II. Related Work

A. Imitation Learning and Corrective Dataset Aggregation

Imitation learning has been widely used for robot manipulation [1–3, 6–8], but behavioral cloning is susceptible to compounding errors. DAgger addresses this issue by iteratively collecting expert labels on policy-induced states and aggregating them into the training set [4], which is costly and not scalable. Prior work tries to reduce the frequency of human-expert queries using safety classifiers [9], novelty estimation [10], and human judgment [11, 12]. Intentionally inducing policy divergence offers another way to collect corrective data more efficiently [13, 14]. However, even with improved approaches, human effort still limits corrective data collection, so corrective data make up only a small portion of the dataset. Therefore, our work uses keypoints and simulation to automate policy divergence detection and corrective data generation, making on-policy corrective data the primary source of training data.

B. Keypoint-Based Motion Planning and Policy Monitoring

Keypoints provide a compact, object-centric representation for specifying manipulation objectives. kPAM first formulates category-level manipulation goals as geometric costs and constraints over semantic 3D keypoints [15]. Recent advances in language models have enabled the generation of executable robot programs from natural-language instructions [16], making keypoints a useful interface for grounding these instructions. Prior work combines Segment Anything with vision-language models to identify consistent task-relevant keypoints across object instances [17], uses keypoint-based affordances to connect vision-language model predictions with robot actions [18], and represents manipulation tasks as programmatic constraints over relational 3D keypoints [19]. ReKep is particularly relevant to our work, as we build on its formulation of keypoint-constrained optimization for motion planning and use of agentic programming.

Beyond motion planning, keypoints also provide a geometric interface for reward design and policy monitoring. Prior work uses keypoint relations to define rewards for reinforcement learning in simulation [20, 21] and utilizes visually tracked 2D keypoints for policy execution monitoring [22]. However, these methods need additional human design in keypoint programs and could not provide stage-wise failure localization for motion planning. Our framework reuses motion-planning constraints and monitors the policy based on ground-truth simulator keypoints. It then compares policy trajectories with planner references to locate divergent steps. The identified failure states and remaining stage constraints are then used to guide corrective planning, producing targeted demonstrations for iterative policy fine-tuning.

C. Simulation for Data Generation and Policy Evaluation

Prior work uses physics-based simulation [23–26] to scale robot demonstration collection by retargeting source trajectories, learning or optimizing expert behaviors, and synthesizing tasks and controllers with language or vision-language models [5, 27–34]. Although these methods substantially broaden data coverage, their demonstrations are generally generated independently of failures encountered by the evolving policy. Simulation has also been used for policy evaluation [35–37]. However, these evaluation signals are typically used for benchmarking or assessing policy performance rather than to collect corrective demonstrations from the corresponding failure states.Our framework bridges data generation and policy evaluation in an iterative loop: keypoint programs generate the initial demonstrations, monitor policy rollouts, and localize divergence; detected failures then trigger targeted corrective data generation from the associated simulator states.

After the policy achieves near-perfect performance in simulation, domain randomization is applied. Following prior work [8, 38–44], we randomize camera extrinsics and intrinsics, object appearance, background color and lighting, and fine-tune the policy on the domain randomized data to enable zero-shot sim-to-real transfer.

III. Method

As shown in Fig. 2, UniKey consists of four modules: (a) keypoint programs, (b) automatic failure detection, (c) auto-

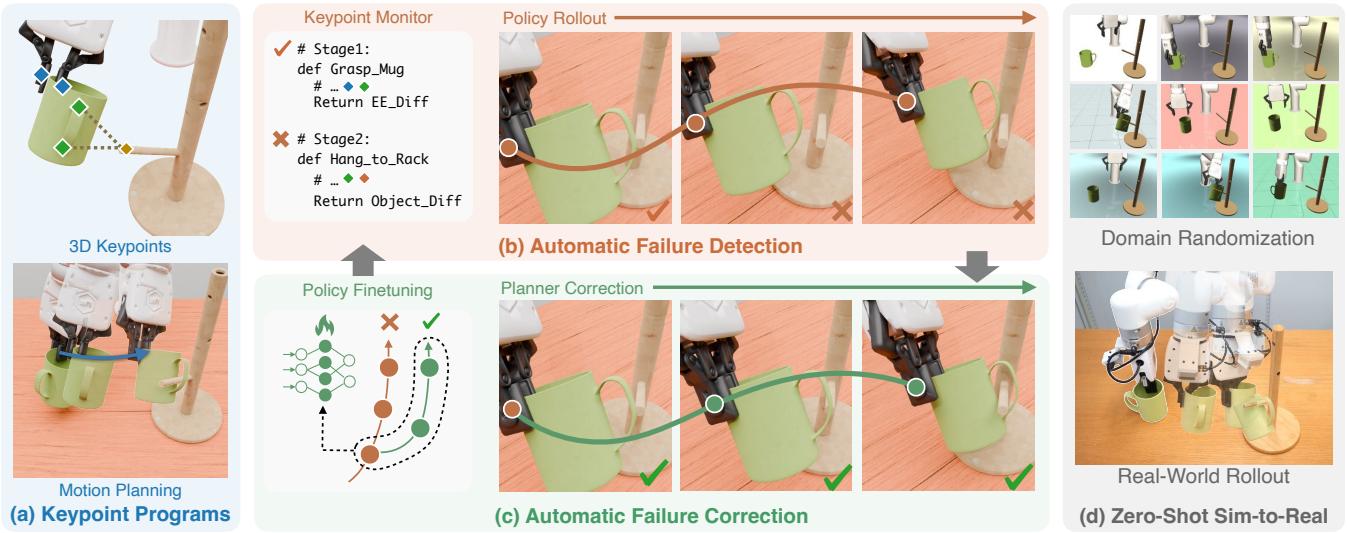


Fig. 2: **UniKey Overview.** (a) A vision-language-model agent generates keypoint programs that enable motion planning based on object keypoints. (b) The imitation learning policy is monitored stage-wise using keypoint programs and step-wise using end-effector keypoint comparisons between the planner and the policy. (c) The planner takes over from the policy state at the detected divergence frame and generates corrective data for policy finetuning. The policy achieves near-perfect performance in simulation through iterations of (b) and (c). (d) The policy is finetuned on the domain-randomized dataset and achieves robust zero-shot performance in the real world.

matic failure correction, and (d) zero-shot sim-to-real. We describe each module in detail in the following subsections.

A. Keypoint Programs

We represent the manipulation task as a constrained optimization problem of keypoints. Each keypoint is a 3D point attached to a rigid body, an articulated link, or a particle. Let $\mathbf{k}_i^t \in \mathbb{R}^3$ denote the 3D coordinates of keypoint i at timestep t, and let $\mathbf{k}^t \in \mathbb{R}^{3N}$ denote the concatenated coordinates of all N keypoints. We decompose a task into L stages. Each stage's objective is denoted as constraint functions: $F_l(\mathbf{k}) \leq \mathbf{0}$, , where $F_l : \mathbb{R}^{3N} \to \mathbb{R}^{m_l}$ is a vector-valued constraint function and m_l is the number of constraints in stage l, for $l = 1, \ldots, L$

The optimization variable $T \in SE(3)$ is a relative rigid transformation applied in the world frame, where $R \in SO(3)$ and $\mathbf{p} \in \mathbb{R}^3$ denote its rotation and translation, respectively. Let G denote the indices of keypoints currently attached to the end effector or the grasped object. The predicted keypoint coordinates are

$$\mathbf{k}_i^{t_{l+1}} = \begin{cases} R\mathbf{k}_i^{t_l} + \mathbf{p}, & i \in G, \\ \mathbf{k}_i^{t_l}, & \text{otherwise.} \end{cases} \tag{1}$$

The optimization problem is written as

$$\begin{aligned} T^* = \arg\min_T \; & ||T T_{\text{ee}}^{t_l} - T_{\text{ee}}^{t_l}||_F^2 \\ \text{s.t.} \quad & F_{l+1}\left(\mathbf{k}^{t_{l+1}}\right) \leq 0, \end{aligned} \tag{2}$$

where $T_{\text{ee}}^{t_l}$ is the end effector pose at the end timestep of stage l, $|\cdot|_F$ denotes the Frobenius norm. The optimized relative transformation T^* yields the target end-effector pose $T^* T_{\text{ee}}^{t_l}$, satisfying the stage constraints while remaining close to the current end-effector pose.

We solve Eq. 2 using dual annealing [45] with SLSQP [46] stage by stage, where the simulator states, image observations and planner actions are recorded $\tau_{\text{planner}} = \{(s^t, o^t, a^t)\}_t$.

For brevity, we denote planning from state s^t through the remaining stages $l, \ldots, L$ as $\text{Plan}(s^t, F_{l:L})$.

The planner can generate both *complete* and *corrective* demonstrations using the same keypoint program. To generate a complete demonstration, we sample an initial state s^0 from the task initialization distribution and execute the keypoint program from the first stage:

$$\tau^{\text{complete}} = \text{Plan}\left(s^0, F_{1:L}\right). \tag{3}$$

Repeating this process produces the dense sampling demonstration dataset.

For corrective demonstration generation, the planner takes over from an intermediate state encountered during policy execution. Given a policy rollout state s^t and its corresponding task stage l_t, we reset the simulator to s^t and generate a corrective trajectory of H steps:

$$\tau^{\text{correction}} = \text{Plan}\left(s^t, F_{l_t:L}, H\right). \tag{4}$$

These corrective trajectories are added to the demonstration dataset, providing expert correction from policy-induced states.

B. Automatic Failure Detection

Although the programmatically generated dataset provides high-quality expert demonstrations, the learned policy π_θ may encounter out-of-distribution states during closed-loop execution. We therefore evaluate the policy in simulation and reuse the relational keypoint program to automatically monitor each policy rollout.

The monitor operates at two levels. The stage-wise monitor evaluates the stage constraints to identify the stage at which the policy fails, while the step-wise monitor compares the policy trajectory with a planner-generated reference trajectory to identify the first timestep at which the deviation exceeds a predefined threshold.

Algorithm 1 UNIKEY Training Procedure

Require: Stage constraints $F_{1:L}$
Require: Divergence threshold δ; corrective trajectory length H
Require: Aggregation rounds J; expert-data budget per round B
Generate initial dataset $\mathbf{D}_0$ using $F_{1:L}$
$\pi_{\theta_0} \leftarrow \text{Train}(\mathbf{D}_0)$
for $j = 0, \ldots, J-1$ **do**
  $\mathbf{C}_{j+1} \leftarrow \emptyset$
  while $|\mathbf{C}_{j+1}| < B$ **do**
    $\hat{\tau} \leftarrow \text{Rollout}(\pi_{\theta_j})$
    if $\hat{\tau}$ fails **then**
      $(t_{\text{div}}, l) \leftarrow \text{DetectDivergence}(\hat{\tau}, F_{1:L}, \delta)$
      $\tau^{\text{correction}} \leftarrow \text{Plan}(\hat{s}^{t_{\text{div}}}, F_{l:L}, H)$
      $\mathbf{C}_{j+1} \leftarrow \mathbf{C}_{j+1} \cup \tau^{\text{correction}}$
    end if
  end while
  $\mathbf{D}_{j+1} \leftarrow \mathbf{D}_j \cup \mathbf{C}_{j+1}$
  $\pi_{\theta_{j+1}} \leftarrow \text{Finetune}(\pi_{\theta_j}, \mathbf{D}_{j+1})$
end for
return $\text{Finetune}(\pi_{\theta_J}, \text{Randomize}(\mathbf{D}_J))$

1) Stage-Wise Monitor: Consider a policy rollout

$$\hat{\tau} = ((\hat{s}^0, \hat{o}^0, \hat{a}^0), \ldots, (\hat{s}^{t_{\text{end}}}, \hat{o}^{t_{\text{end}}}, \hat{a}^{t_{\text{end}}})).$$

We evaluate the stage-wise constraint functions on the keypoints embedded in simulator state $F_{1:L}$, introduced in Sec. III-A, from $t = 0$ to the maximum task completion time $t = t_{\text{end}}$. If a timestep at which the constraints are satisfied exists, we consider the rollout successful at stage l and continue to the next stage. Otherwise, the rollout is considered to have failed. The rollout is considered successful only if all stages are completed sequentially.

2) Step-Wise Monitor: Stage-Wise granularity alone is insufficient. Policy divergence does not always occur at the beginning of a stage and instead often occurs near its completion. For example, in the hang-mug task, the last stage is to hang the mug onto the rack. The policy is most likely to diverge from the planner when the mug is nearly placed onto the rack, rather than during the early-stage motion.

Given a policy rollout that fails at stage l, we locate the divergence timestep within that stage using a planner-generated reference trajectory. Let t_{start} denote the end time of the last successful stage. We reset the simulator to the corresponding policy state $\hat{s}^{t_{\text{start}}}$ and execute the remaining motion planning from stage l to generate a reference trajectory.

We compare the policy and planner end-effector trajectories using three keypoints on the end effector. Let $\mathbf{k}^t_{\text{policy,EE}} \in \mathbb{R}^9$ and $\mathbf{k}^t_{\text{planner,EE}} \in \mathbb{R}^9$ denote their concatenated 3D coordinates at timestep t along the policy and planner trajectories, respectively. The deviation at timestep t is defined as

$$d^t = \left\| \mathbf{k}^t_{\text{policy,EE}} - \mathbf{k}^t_{\text{planner,EE}} \right\|_2. \tag{5}$$

The divergence timestep t_{div} is the first timestep within the failed stage at which d^t exceeds the divergence threshold δ. We record this timestep and its corresponding policy state $\hat{s}^{t_{\text{div}}}$ for corrective data generation.

C. Automatic Failure Correction

Here, we utilize the motion planning in Sec. III-A with the policy failure monitor in Sec. III-B to automatically construct corrective data. After the policy failure monitor identifies the earliest divergent step in the failed stage, we use the from-the-middle capability of the keypoint motion planner to correct the failure. Specifically, the simulator is reset to the recorded policy state $\hat{s}^{t_{\text{div}}}$, and the planner executes the remaining keypoint program from the failed stage: $\tau^{\text{correction}} = \text{Plan}\left(\hat{s}^{t_{\text{div}}}, F_{l_{t_{\text{div}}}:L}, H\right)$. The planner therefore takes over directly from the policy-induced OOD state and generates a corrective trajectory toward task completion.

Corrective Data Length. For each detected failure, we use the planner to generate an H-step corrective trajectory starting from the identified policy failure state. The corrective horizon H is a key hyperparameter that we study in Sec. IV-D. We repeat this process until a total of B expert action steps are generated for the current aggregation round.

Iterative Dataset Aggregation. Let $\mathbf{D}_j$ denote the aggregated dataset after iteration j. At each iteration, we allocate a fixed expert-data budget of B action steps. We repeatedly evaluate the current policy and generate H-step corrective trajectories from detected failure states until the per-round budget is reached. Let $\mathbf{C}_{j+1}$ denote the set of corrective trajectories collected in iteration $j+1$, with $|\mathbf{C}_{j+1}| = B$ expert action steps. The dataset is updated as

$$\mathbf{D}_{j+1} = \mathbf{D}_j \cup \mathbf{C}_{j+1}. \tag{6}$$

Starting from the previous policy checkpoint π_{θ_j}, we finetune the policy on the complete aggregated dataset $\mathbf{D}_{j+1}$ to obtain $\pi_{\theta_{j+1}}$. We use all accumulated data to help prevent catastrophic forgetting.

D. Zero-shot Sim-to-Real

After iterative corrective data generation and policy finetuning, we obtain a policy that achieves near-perfect performance in simulation. To reduce gaps in rendering, camera calibration, and physical dynamics, we augment the aggregated dataset by randomizing the visual domain, camera parameters, and physical dynamics in simulation. Domain randomization perturbs lighting, textures, object appearance, and backgrounds; camera randomization perturbs intrinsic and extrinsic parameters around the calibrated real-camera configuration; and dynamics randomization perturbs physical properties such as contact parameters, mass, friction, and damping. We then fine-tune the simulation policy on the augmented dataset and deploy it directly on the physical robot without real-world demonstrations or real-world policy fine-tuning, enabling zero-shot sim-to-real transfer.

We summarize the UNIKEY training procedure in Algorithm 1. The policy is first trained on an initial dataset from dense sampling. In each round, UNIKEY rollouts the policy and uses keypoints to automatically detect failed stage and divergence frame and generate corrective data. Then it aggregates the corrective data into the dataset and finetunes the policy. Finally, the policy is finetuned on the augmented dataset for sim-to-real transfer.

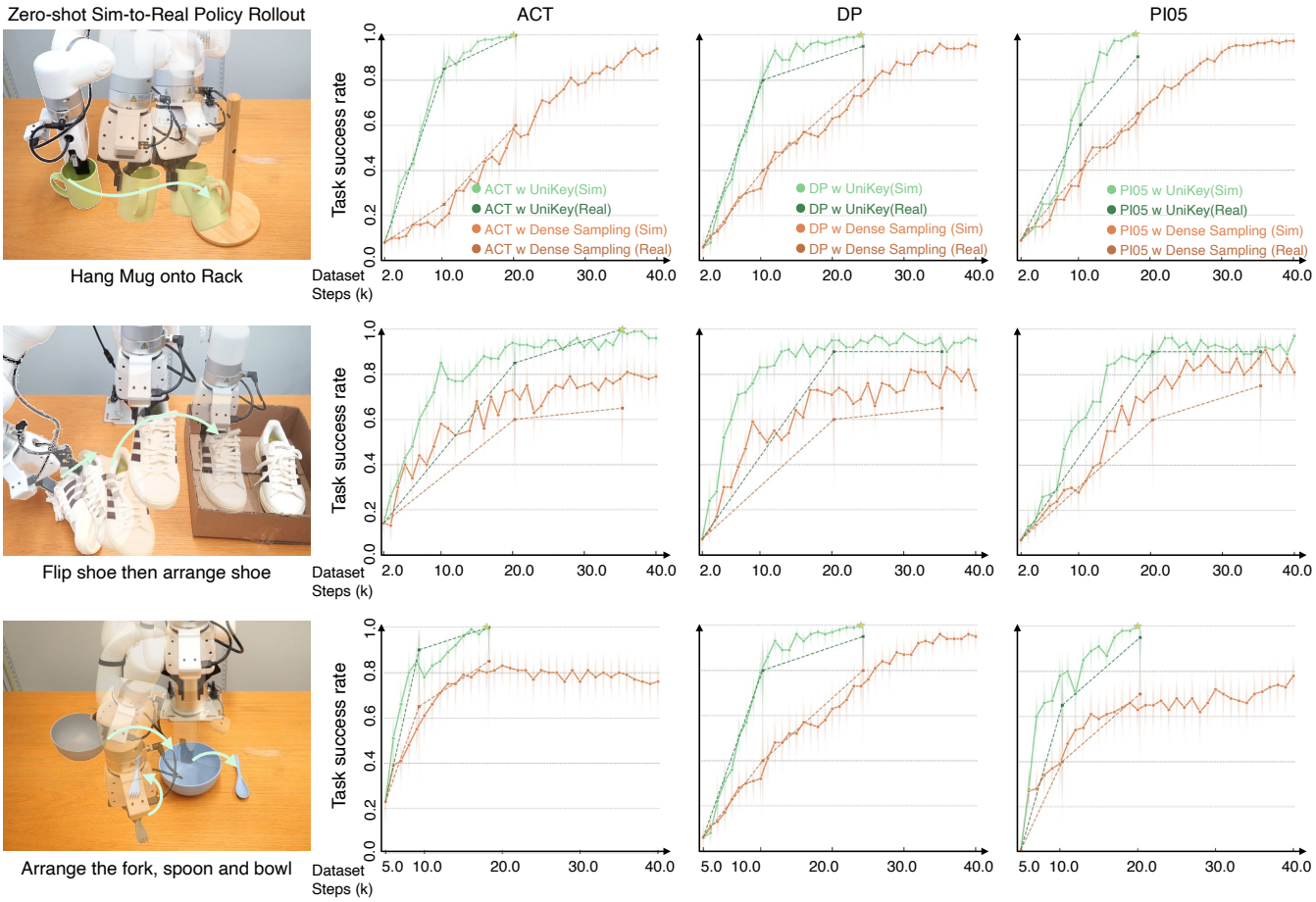


Fig. 3: **Quantitative evaluation of sample efficiency and task success rates across tasks and policy architectures.** Performance comparison between baseline dense sampling (brown) and UNIKEY (green) across three manipulation tasks (rows: mug hanging, shoe flipping and arranging and table arranging) and three policy architectures (cols: ACT [1], Diffusion Policy [2], and $\pi_{0.5}$ [3]). Both dense-sampling and UNIKEY initialize from identical base checkpoints trained on the same dense samplingdataset. At each new round, newly collected complete expert data (dense sampling) / corrective data (UNIKEY) are aggregated into the datasets respectively and the previous round's checkpoint is finetuned on the whole new dataset. Our approach achieves significantly higher sample efficiency with nearly perfect performance in both simulation and the real world.

IV. EXPERIMENTS

In this section, we study three questions: (1) How does our method compare with baselines in terms of data efficiency and final success rate in simulation and the real world (Section IV-B)? (2) What are the qualitative and quantitative characteristics of the collected corrective data (Section IV-C)? (3) How does the performance of our framework vary with the corrective-data horizon and policy-divergence threshold (Section IV-D)?

A. Experiment Setup

a) Task Setup: We evaluate our method on thirteen robotic manipulation tasks. Three tasks are instantiated in both mujoco simulation [23] and real world: (1) hang the mug onto the rack, (2) flip and then arrange the shoe, and (3) arrange the spoon, fork and bowl sequentially. The real-world embodiment is an xArm 7 robot arm (UFACTORY). We additionally evaluate our method on ten simulation tasks from the RoboTwin benchmark [5]. For the three sim-to-real tasks, all training data are generated entirely in simulation while the resulting policies are evaluated in both simulation and the real world.

b) Imitation-Learning Policies: Our contribution is a data-generation and aggregation strategy rather than a new policy architecture. We therefore evaluate our method with multiple visuomotor policies, including Action Chunking with Transformers (ACT) [1], Diffusion Policy (DP) [2], and $\pi_{0.5}$ [3]. For each policy architecture, our method and the dense-sampling baseline start from the same initial training dataset and checkpoint and add the same amount of new data in each round.

c) Evaluation Protocol: We measure the expert-data budget by the number of expert action steps stored in the training dataset and compare methods under matched cumulative budgets. Both methods start from the same initial dataset $\mathbf{D}_0$ and policy checkpoint. In each aggregation round, both methods receive the same additional expert-data budget of B action steps. Dense sampling allocates this budget to complete planner demonstrations, while UNIKEY allocates it to corrective data from policy-induced failure states. The newly collected data are aggregated with the existing dataset, and the previous checkpoint is fine-tuned on the updated dataset. We report task success rates over independently

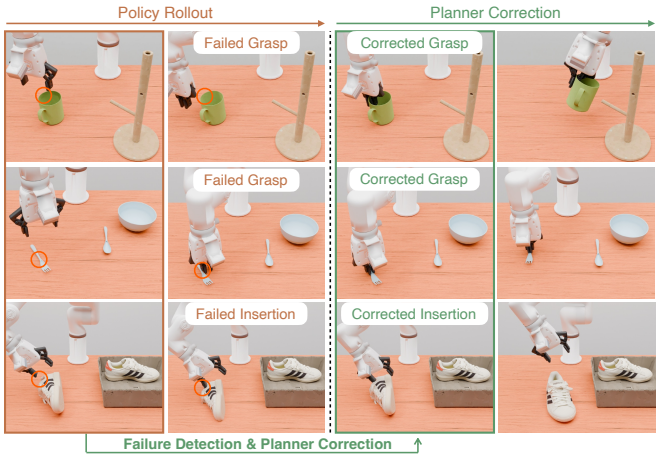


Fig. 4: **Qualitative examples of policy failures and planner corrections.** The left two columns show policy rollout failures monitored by keypoints. The right two columns show corrective motions generated by the keypoint planner. *Top row:* The policy mis-predicts the mug rim position and fails to grasp the mug. *Middle row:* The policy fails to grasp the fork due to the prediction error on the fork grasp point. *Bottom row:* The policy fails to insert the gripper into the shoe due to insertion pose prediction error. In each row, UNIKEY successfully detects policy failed stage and frame and generates corresponding corrective data.

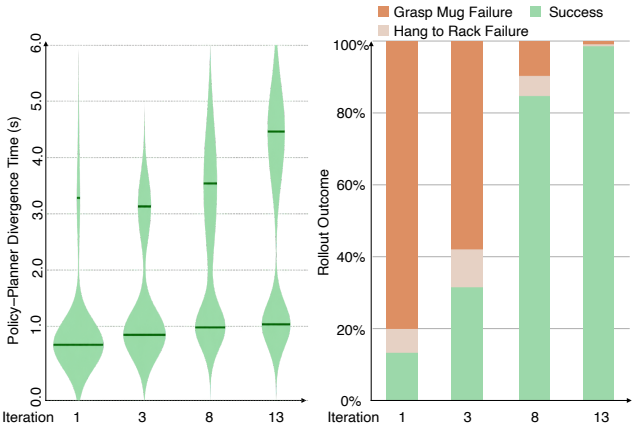


Fig. 5: **Left: Evolution of Divergence Time.** Distribution of the policy–planner divergence time across corrective-data aggregation iterations. As training progresses, the distribution, fitted with a two-component Gaussian mixture model (GMM, $K = 2$), shifts toward later timesteps, indicating that the policy follows the expert planner trajectory for a longer duration before diverging. **Right: Evolution of Failure Modes.** Distribution of rollout outcomes across iterations. Early rollouts are dominated by grasp failures. As corrective data are iteratively aggregated, the success rate increases and both types of failure become rare.

sampled evaluation scenes, using 100 rollouts in simulation and 20 rollouts in the real world.

B. Data Efficiency and Final Success Rate

Figure 3 compares our corrective-data method with dense sampling under matched cumulative data budgets. Across the evaluated tasks, policies trained with our method generally improve more rapidly as the dataset grows. In particular, our method concentrates supervision near states at which the current policy fails, whereas dense sampling continues to allocate data to both solved and unsolved portions of the task distribution. Consequently, the corrective-data strategy reaches high task success using fewer dataset steps. We quantify data efficiency by the dataset steps required to first reach 70%, 80%, and 90% success, and report the average dense-to-UNIKEY data ratio across these thresholds, tasks, and policy architectures. By this measure, UNIKEY is 2.65× more data-efficient than dense sampling. Furthermore, our method provides targeted corrective supervision at policy-induced states that are poorly covered by the training data, enabling higher final success rates. The improvement is observed across ACT, Diffusion Policy, and $\pi_{0.5}$, indicating that the benefits of our data-collection strategy are consistent across the evaluated policy architectures. For the three sim-to-real tasks, these gains are also observed in the real world. This result suggests that the collected corrections address task-relevant failure modes rather than merely exploiting simulator-specific trajectories.

C. Corrective Data Analysis

a) Representative corrective demonstrations.: Figure 4 visualizes representative policy failures and the corresponding planner-generated corrections. Each row shows a nominal policy rollout up to a detected failure, followed by the corrective demonstration generated after restoring the environment to a recoverable state.

b) Evolution of Divergence TIme and Failure mode: Figure 5 examines how policy behavior evolves over successive corrective-data aggregation iterations. In the left panel, for each iteration, we fit a two-component Gaussian mixture model (GMM, $K = 2$) to the policy–planner divergence times from 100 failed rollout samples. The earlier component mainly corresponds to divergence during mug grasping, whereas the later component corresponds to divergence during the rack-hanging interaction. As corrective data are iteratively aggregated, the distribution becomes increasingly weighted toward the later component. This indicates that the policy progressively learns to complete the earlier grasping motion and follows the planner for a longer duration before diverging during the more precise terminal interaction. The right panel further categorizes the rollout outcomes into grasp failure, rack-hanging failure, and success. In the early iterations, failures are dominated by unsuccessful grasps. After corrective demonstrations are collected from these states, grasp failures decrease substantially and the success rate increases. The overall failure rate decreases substantially over iterations, with only a small fraction of rollouts failing in the final iteration shown. Together, these results show that our method progressively resolves early failure modes and shifts the remaining policy errors toward later, more challenging portions of the task.

D. Divergence Threshold and Corrective Horizons

Figure 6 studies the sensitivity of UNIKEY to two key hyperparameters: the corrective horizon and divergence threshold. The corrective horizon determines the length of the planner-generated corrective trajectory, while the divergence

TABLE I: **Per-task success rates (%) on 10 RoboTwin tasks.** Initial Success Rate denotes the success rate of the policy trained on the initial dataset. UNIKEY reports the best success rate achieved by our method. The numbers in parentheses below the UNIKEY success rates indicate the generated data budgets in steps. Dense Sampling is evaluated using the same data budget with UNIKEY for each task. On the RoboTwin benchmark tasks [5], UNIKEY achieves significantly higher success rates than dense sampling.

Method	Adjust Bottle	Beat Block w/ Hammer	Rank Blocks by Color	Rank Blocks by Size	Click Alarm Clock	Click Bell	Dump Bin into Big Bin	Move Can to Pot	Move Playing Card Away	Move Stapler to Pad
Initial Success Rate	45	58	7	46	8	16	70	25	18	0
Dense Sampling	87	95	60	93	75	81	87	87	72	39
UNIKEY	**100** (5k)	**100** (7k)	**91** (20k)	**100** (18k)	**100** (15k)	**100** (10k)	**100** (3k)	**100** (10k)	**100** (9k)	**91** (28k)

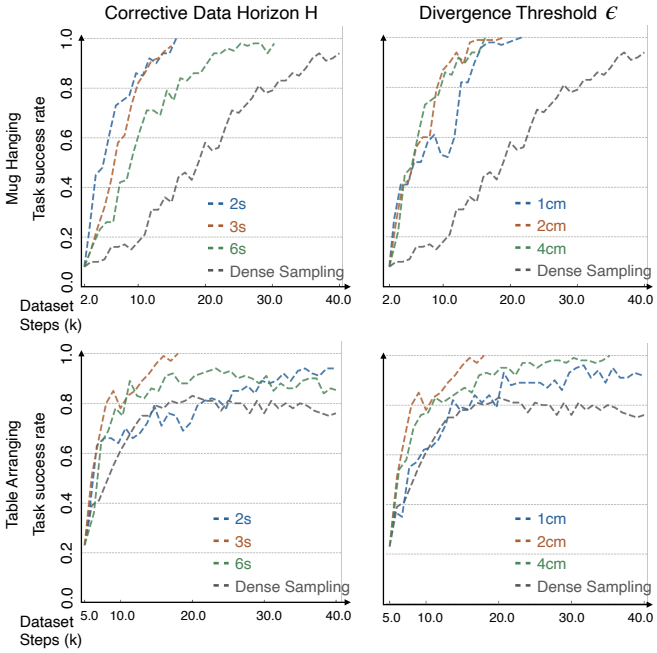


Fig. 6: **Study of the corrective horizon and divergence threshold.** Task success rate as a function of cumulative dataset size (in thousands of steps) on Mug Hanging (top) and Table Arranging (bottom). Left: comparison of corrective horizons of 2, 3, and 6 s. Right: comparison of divergence thresholds of 1, 2, and 4 cm. Gray dashed curves represent the dense sampling baseline. Empirically, we find that a 2 / 3 s corrective horizon and a 2 cm divergence threshold provide the best overall performance.

threshold controls how far the policy may deviate from the planner reference before a correction is generated. We vary each parameter independently while holding the other fixed.

Across both *Mug Hanging* and *Table Arranging*, corrective data consistently improve more rapidly than dense sampling over a broad range of settings. For *Mug Hanging*, corrective horizons of 2–3 s achieve similar rapid convergence, while a 6 s horizon requires more data. For *Table Arranging*, a 3 s horizon yields the most stable improvement. This reflects a trade-off in correction length: overly short segments may not capture sufficient recovery behavior, whereas overly long segments spend additional data on portions of the trajectory that the policy can already execute.

A similar trade-off appears for the divergence threshold. A small threshold may generate corrections for benign deviations, resulting in redundant supervision, while a large threshold delays correction until the policy has deviated farther from the planner trajectory. Across both tasks, an intermediate threshold of 2 cm provides consistently strong performance. Overall, the method remains effective across the tested settings, suggesting that its gains are not sensitive to a narrowly tuned hyperparameter configuration.

V. CONCLUSION

We presented UNIKEY, an automated framework that uses keypoints as a unified interface for policy monitoring and motion planning. By identifying failed stages and divergence timesteps, the framework resets the simulator to the corresponding policy states and generates targeted corrective demonstrations for iterative dataset aggregation and policy finetuning. Experiments on three sim-to-real manipulation tasks across three policy architectures, together with evaluations on ten RoboTwin tasks, demonstrate higher sample efficiency and task success rates than dense sampling of complete expert trajectories. Our analysis further shows that iterative correction reduces early-stage failures and delays policy divergence. Combined with simulation randomization, the learned policies support zero-shot real-world deployment without real-world demonstrations or policy finetuning.

Limitations and Future Work. Our framework relies on a simulator that sufficiently captures task-relevant geometry and dynamics, as well as rigid-body assumptions in keypoint-based planning. Future work could explore action-conditioned video models as alternative simulators for corrective data generation. Another direction is to use policies trained with model-free reinforcement learning as alternative experts for generating corrective demonstrations.

REFERENCES

[1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.

[2] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[3] P. Intelligence et al., "$\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[4] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[5] T. Chen et al., "Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[6] K. Black et al., "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[7] P. Intelligence et al., "$\pi^*_{0.6}$: A vla that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[8] J. Mercat et al., "VLA Foundry: A unified framework for training vision-language-action models," Toyota Research Institute, Tech. Rep., 2026.

[9] J. Zhang and K. Cho, "Query-efficient imitation learning for end-to-end simulated driving," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, 2017, pp. 2891–2897.

[10] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, "ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164, PMLR, 2022, pp. 598–608.

[11] K. Menda, K. Driggs-Campbell, and M. J. Kochenderfer, "Ensembledagger: A bayesian approach to safe imitation learning," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2019, pp. 5041–5048.

[12] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, pp. 8077–8083.

[13] X. Zhang, M. Chang, P. Kumar, and S. Gupta, "Diffusion meets dagger: Supercharging eye-in-hand imitation learning," *arXiv preprint arXiv:2402.17768*, 2024.

[14] G. Jiang et al., "Gsworld: Closed-loop photo-realistic simulation suite for robotic manipulation," *arXiv preprint arXiv:2510.20813*, 2025.

[15] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, "Kpam: Keypoint affordances for category-level robotic manipulation," in *The International Symposium of Robotics Research*, Springer, 2019, pp. 132–157.

[16] J. Liang et al., "Code as policies: Language model programs for embodied control," in *2023 IEEE International conference on robotics and automation (ICRA)*, IEEE, 2023, pp. 9493–9500.

[17] X. Fang et al., "Kalm: Keypoint abstraction using large models for object-relative imitation learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 8307–8314.

[18] F. Liu, K. Fang, P. Abbeel, and S. Levine, "Moka: Open-world robotic manipulation through mark-based visual prompting," *arXiv preprint arXiv:2403.03174*, 2024.

[19] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, "Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation," *arXiv preprint arXiv:2409.01652*, 2024.

[20] Y. Narang et al., "Factory: Fast contact for robotic assembly," *arXiv preprint arXiv:2205.03532*, 2022.

[21] S. Patel et al., "A real-to-sim-to-real approach to robotic manipulation with vlm-generated iterative keypoint rewards," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 8258–8266.

[22] E. Zhou et al., "Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2025, pp. 6919–6929.

[23] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*, IEEE, 2012, pp. 5026–5033.

[24] NVIDIA, *Nvidia isaac sim*, `https://developer.nvidia.com/isaac-sim`, 2024.

[25] F. Xiang et al., "SAPIEN: A simulated part-based interactive environment," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2020.

[26] V. Makoviychuk et al., "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[27] A. Mandlekar et al., "Mimicgen: A data generation system for scalable robot learning using human demonstrations," *arXiv preprint arXiv:2310.17596*, 2023.

[28] C. Garrett, A. Mandlekar, B. Wen, and D. Fox, "Skillmimicgen: Automated demonstration generation for efficient skill learning and deployment," *arXiv preprint arXiv:2410.18907*, 2024.

[29] Z. Jiang et al., "Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 16 923–16 930.

[30] L. Wang et al., "Gensim: Generating robotic simulation tasks via large language models," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 4890–4924.

[31] P. Hua et al., "Gensim2: Scaling robot data generation with multi-modal and reasoning llms," *arXiv preprint arXiv:2410.03645*, 2024.

[32] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu, "Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning," *arXiv preprint arXiv:2502.16932*, 2025.

[33] L. Yang et al., "Physics-driven data generation for contact-rich manipulation via trajectory optimization," *arXiv preprint arXiv:2502.20382*, 2025.

[34] C. Pan et al., "Spider: Scalable physics-informed dexterous retargeting," *arXiv preprint arXiv:2511.09484*, 2025.

[35] K. Zhang et al., "Real-to-sim robot policy evaluation with gaussian splatting simulation of soft-body interactions," *arXiv preprint arXiv:2511.04665*, 2025.

[36] A. Jain et al., *Polaris: Scalable real-to-sim evaluations for generalist robot policies*, 2025. arXiv: `2512.16881 [cs.RO]`.

[37] J. Yu et al., *Real2render2real: Scaling robot data without dynamics simulation or robot hardware*, 2025. arXiv: `2505.09601 [cs.RO]`.

[38] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2017, pp. 23–30.

[39] A. Deshpande et al., "Molmob0t: Large-scale simulation enables zero-shot manipulation," *arXiv preprint arXiv:2603.16861*, 2026.

[40] T. He et al., "Viral: Visual sim-to-real at scale for humanoid loco-manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 13 430–13 441.

[41] F. Sadeghi and S. Levine, "Cad2rl: Real single-image flight without a single real image," *arXiv preprint arXiv:1611.04201*, 2016.

[42] I. Akkaya et al., "Solving rubik's cube with a robot hand," *arXiv preprint arXiv:1910.07113*, 2019.

[43] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE international conference on*

robotics and automation (ICRA), IEEE, 2018, pp. 3803–3810.

[44] O. M. Andrychowicz et al., "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[45] Y. Xiang, D. Sun, W. Fan, and X. Gong, "Generalized simulated annealing algorithm and its application to the thomson model," *Physics Letters A*, vol. 233, no. 3, pp. 216–220, 1997.

[46] D. Kraft, "A software package for sequential quadratic programming," *Forschungsbericht- Deutsche Forschungs- und Versuchsanstalt fur Luft- und Raumfahrt*, 1988.

[47] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, et al., *Lerobot: State-of-the-art machine learning for real-world robotics in pytorch*, `https://github.com/huggingface/lerobot`, 2024.

[48] Physical Intelligence, *OpenPI: Open-source models and code for robot learning*, `https://github.com/Physical-Intelligence/openpi`, 2025.

APPENDIX I

A. Platform and Tasks

1) Robot Setup: We use a tabletop-mounted UFactory xArm 7 robot equipped with a standard xArm gripper. Two Intel RealSense RGB-D cameras provide visual observations: a fixed D455 overlooking the workspace and a wrist-mounted D405 attached to link 7 via a custom 3D-printed bracket.

2) Tasks and Success Criteria: We evaluate our method on three manipulation tasks in both simulation and the real world. The task objectives and success criteria are defined as follows.

a) Hang Mug: The robot grasps a mug from the table and hangs it on a rack. The task requires precise alignment of the mug handle with the rack, particularly when the mug is grasped with an offset relative to the gripper. A rollout is successful if the mug remains securely hanging on the rack after the gripper releases it.

b) Arrange Table: The robot sequentially manipulates three objects (a spoon, a fork, and a bowl) to arrange them in designated positions on the table. A rollout is successful if all three objects are placed in their target positions.

c) Flip and Arrange Shoe: Starting with an upside-down shoe, the robot must flip it upright and place it within a designated region inside a shoe box. A rollout is successful if the shoe is upright and positioned within the target region.

3) Object Initialization: For each task, we sample initial object configurations from predefined task-specific ranges, as summarized in Table II. Position and orientation variations are defined relative to nominal object poses. Fig. 7 visualizes the sampled initial configurations for all three tasks. We use the same initialization distributions across all compared methods to ensure a fair evaluation.

TABLE II: Object initialization ranges for the three tasks.

Task	Object	x (cm)	y (cm)	θ (degree)
Hang Mug	Mug	± 5	± 5	± 90
	Rack	± 1	± 2	± 32.5
Arrange Table	Bowl	± 2.5	± 2.5	0
	Fork	± 5	± 5	± 45
	Spoon	± 5	± 5	± 45
Flip Shoe	Right shoe	± 2.5	± 5	± 22.5
	Tray	0	0	$\{-45, 45\}$

Note: Continuous offsets are sampled uniformly. The two tray orientations are sampled with equal probability, and the left shoe remains aligned with the tray.

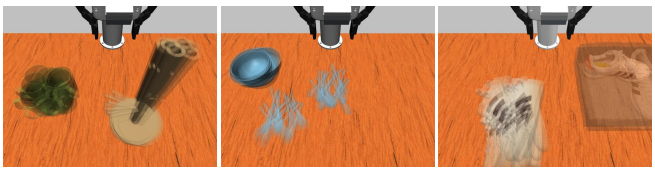


Fig. 7: **Visualization of Initial Object Configurations.** Overlaid initial configurations for *Hang Mug* (left), *Arrange Table* (middle), and *Flip and Arrange Shoe* (right).

B. Datasets

1) Initial Demonstrations: For each task, we initialize policy training with a small set of expert demonstrations generated in simulation. Both UniKey and dense sampling use the same initial dataset and policy checkpoint to ensure a fair comparison.

2) Dataset Aggregation: We iteratively expand the training dataset using two strategies. UniKey generates corrective trajectories when policy divergence is detected, while dense sampling collects complete expert demonstrations. After each data collection round, the newly generated trajectories are added to the training dataset for policy fine-tuning. We compare the two methods at matched cumulative dataset sizes, measured by the total number of action steps.

3) Domain Randomization: To enable sim-to-real transfer, we apply domain randomization to the simulation data, including variations in object appearance, background colors and textures, camera configurations, and lighting conditions. Specifically, we randomize object and background colors through hue, saturation, and brightness adjustments, perturb camera extrinsics and focal lengths, and vary the number, positions, and intensities of light sources. Appearance and lighting parameters are shared across camera views, while camera perturbations are sampled independently.

The randomization parameters and their sampling ranges are summarized in Table III. Representative randomized scenes are shown in Fig. 8. We fine-tune the policies on randomized simulation data before deploying them in the real world, without collecting real-world demonstrations or performing real-world fine-tuning.

C. Policy Training

1) Observations and Actions: We collect RGB observations at 50 Hz and execute the policy at 10 Hz. At each policy step, the latest available visual observations are used together with robot proprioceptive states. The policy predicts end-effector actions consisting of a 3D position, a 6D rotation representation, and a gripper state.

D. Policy Training

1) Observations and Actions: We collect RGB observations from two cameras at 50 Hz, while the policies operate at 10 Hz. At each policy step, the latest available camera observations are used together with robot proprioceptive states. Keypoints are initialized using DINOv3 feature matching and subsequently tracked using CoTracker. Both robot states and actions are represented as 10-dimensional vectors consisting of 3D position, 6D rotation representation, and gripper state.

2) Policy Architectures: We evaluate three imitation learning architectures: Action Chunking Transformer (ACT), Diffusion Policy (DP), and $\pi_{0.5}$. We use the LeRobot [47] implementations for ACT and DP, and the OpenPI [48] implementation for $\pi_{0.5}$. We configure all three architectures to use the same observation and action interfaces, with architecture-specific normalization settings. This allows us to evaluate the effectiveness of automatic corrective data generation across different policy architectures.

TABLE III: Domain randomization configuration shared across all three tasks. Parameters are sampled once per episode.

Component	Parameter	Range / Value
D455 Camera	Translation, each axis (cm)	± 0.5
	Rotation, each axis (°)	± 1.0
	Focal-length multiplier	$[0.99, 1.01]$
D405 Camera	Translation, each axis (cm)	± 0.2
	Rotation, each axis (°)	± 0.5
	Focal-length multiplier	$[0.99, 1.01]$
Table and Floor	Hue shift (°)	± 20
	Saturation multiplier	$[0.7, 1.3]$
	Brightness multiplier	$[0.7, 1.3]$
Object Appearance	Hue shift (°)	± 50
	Saturation multiplier	$[0.7, 1.3]$
	Brightness multiplier	$[0.7, 1.3]$
Lighting	Number of lights	$\{1, 2, 3, 4\}$
	Total brightness	$[0.4, 1.8]$
	Diffuse share	$[0.55, 0.95]$
	Light position x (m)	$[-1.5, 1.5]$
	Light position y (m)	$[-1.5, 1.5]$
	Light position z (m)	$[1.5, 4.0]$

Note: Hue shifts are relative to the original appearance, while saturation and brightness are multiplicative factors. Hue values wrap periodically, and saturation and brightness are clamped to $[0, 1]$.

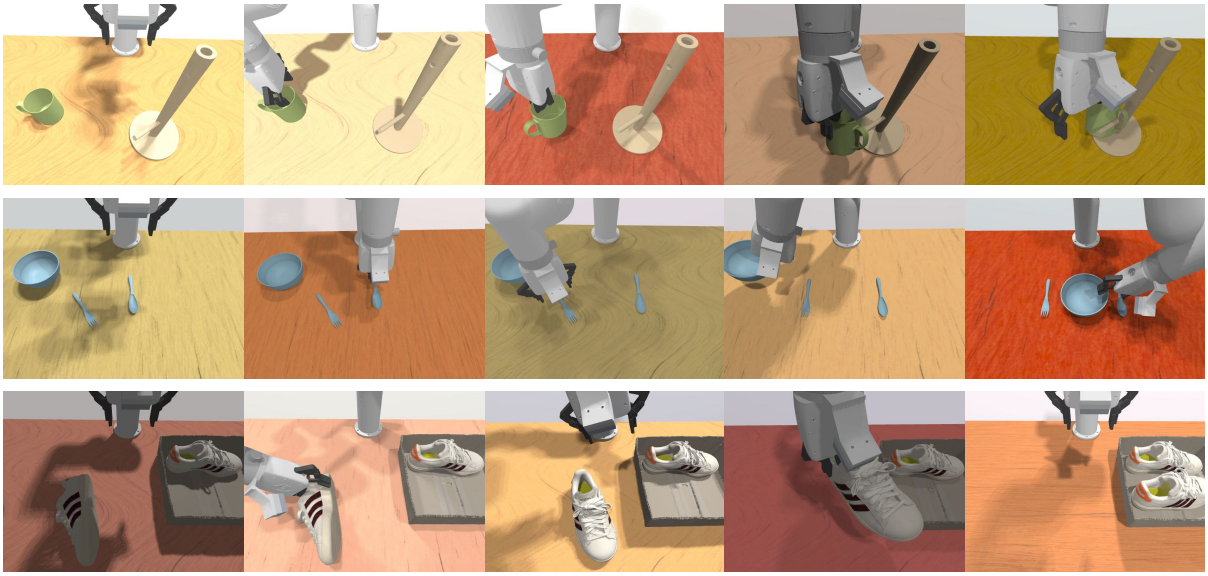


Fig. 8: **Visual Domain Randomization.** Representative examples of randomized simulation environments across the three manipulation tasks. We randomize object appearance, background colors and textures, camera parameters, and lighting conditions to increase visual diversity and facilitate sim-to-real transfer. Object geometry and collision properties remain unchanged.

3) Training and Execution Settings: All policies receive two RGB camera views at a resolution of 224×224 and predict action sequences with a horizon of 16 steps. During deployment, the policies operate at 10 Hz, while low-level robot control runs at 50 Hz. We execute one policy interval before replanning, corresponding to five low-level control updates. We configure the input and output dimensions, normalization schemes, and action horizons for each implementation as summarized in Table IV. For each architecture, UniKey and dense sampling use identical training configurations and differ only in their data generation strategies.

E. Policy Evaluation

1) Evaluation Protocol: We evaluate policy performance in both simulation and the real world. For each task, initial object configurations are sampled from predefined ranges to assess robustness to variations in object placement.

For each policy checkpoint, we conduct 100 rollouts in simulation and 20 rollouts in the real world. We report the success rate, defined as the fraction of rollouts that successfully complete the task.

During real-world deployment, the policy operates at 10 Hz using a non-blocking control pipeline. RGB observa-

TABLE IV: **Policy training and execution settings.** MM denotes min–max normalization, while MM_{xyz} normalizes only the three position channels. All policies use absolute actions.

Setting	**ACT**	**DP**	$\pi_{0.5}$
Implementation	LeRobot	LeRobot	OpenPI
Visual normalization	MM	MM	Identity†
State normalization	MM	MM	MM
Action normalization	MM	MM	MM
Relative actions	No	No	No
Image resolution	224×224	224×224	224×224
Camera views	2	2	2
State dimension	10	10	10
Action dimension	10	10	10
Observation steps	1	1	1
Prediction horizon T_{p}	16	16	16
Execution horizon T_{e}	1	1	1
Policy frequency (Hz)	10	10	10
Action ensembling coefficient	0.02	0.02	0.02
Low-level Control frequency (Hz)	1000	1000	1000

Note: State and action vectors contain position (3), rotation (6), and gripper state (1). $\pi_{0.5}$ internally pads these vectors to 32 dimensions. T_{p} denotes the prediction horizon at 10 Hz, while T_{e} denotes the number of policy intervals executed before replanning. Each policy interval corresponds to five control updates at 50 Hz. †Identity denotes dataset-level normalization; OpenPI performs internal image preprocessing.

tions are collected at 50 Hz, and the policy uses the most recent available observations at each control step. Sensor acquisition, policy inference, and robot control are executed asynchronously to reduce control latency.

Initial Scene Overlay. We first mount the external camera in a fixed position and estimate its extrinsic parameters using a calibration board. We use these parameters to configure the simulated camera pose and train policies in simulation. During real-world evaluation, we overlay the simulated initial scene onto the real camera image and adjust the physical objects to match their simulated placements. We use the same reference configurations across policy checkpoints to ensure consistent initial conditions for real-world comparisons.

Sim-to-Real Comparison. We train policies in simulation with domain randomization and evaluate them on the physical robot. Figure 9 presents representative simulation and real-world rollouts for all three tasks: *Hang Mug*, *Arrange Table*, and *Flip & Arrange Shoe*. The paired sequences show corresponding manipulation stages from both camera views, illustrating policy transfer despite differences in lighting, object appearance, and rendering between simulation and the real world.



Fig. 9: **Sim-to-real rollout comparison.** Representative simulated and real-world executions of *Hang Mug*, *Arrange Table*, and *Flip & Arrange Shoe*, shown from top to bottom. For each task, the simulated sequence is followed by its real-world counterpart. Within each sequence, external-camera views appear above wrist-camera views. Time progresses from left to right, showing corresponding manipulation stages across sim and real.